Gestor de tareas con dos contenedores Docker

Entrega 1 Semana 3

Politécnico Grancolombiano
Énfasis Profesional I Integración Continua

Integrantes: [Completar nombres]
Docente: [Completar nombre]
Fecha: 10 de noviembre de 2026

Introducción

Este proyecto implementa un gestor de tareas para aplicar el uso de Docker a una solución de software. La aplicación permite registrar, consultar y eliminar pendientes. Su arquitectura separa la interfaz web y el almacenamiento en dos servicios que deben comunicarse dentro de una red de Docker.

La guía de la entrega 1 solicita un proyecto en GitHub y dos contenedores comunicados. Se incluye el código, la configuración reproducible y las pruebas realizadas. Las evidencias locales se identifican por separado de las comprobaciones de contenedores.

Objetivo general

Implementar un gestor de tareas mediante dos servicios Docker conectados y documentar su construcción y verificación.

Objetivos específicos

- Organizar el código fuente y sus archivos de configuración en GitHub.
- Definir una imagen para la aplicación web y un servicio independiente de almacenamiento.
- Comprobar las operaciones de la aplicación y verificar la comunicación y persistencia en el entorno de ejecución.

Alcance

La aplicación es una demostración académica local. Incluye tareas de hasta 120 caracteres, validación de entradas y una consulta de salud. No incorpora cuentas de usuario. Jenkins corresponde a una entrega posterior según la guía del módulo.

Fundamentos y arquitectura

Una imagen contiene el sistema de archivos y la configuración necesarios para crear un contenedor. El contenedor es una instancia de esa imagen en ejecución. El Dockerfile define cómo construir la imagen de la aplicación; Docker Compose declara los servicios y sus recursos en un archivo YAML.

Los contenedores comparten el núcleo del sistema donde se ejecutan, mientras que una máquina virtual dispone de un sistema operativo invitado. En este proyecto se utiliza Docker para separar la aplicación y Redis, sin crear una máquina virtual por componente.

Elemento	Tecnología	Responsabilidad
web	Python 3.12, Flask y Gunicorn	Atender solicitudes HTTP y procesar las tareas.
datos	Redis 7.4 Alpine	Guardar las tareas como registros de una estructura hash.
proyecto	Red bridge	Conectar los servicios y permitir resolver sus nombres.
tareas	Volumen Docker	Conservar los archivos de persistencia de Redis.

Recorrido de una solicitud

El navegador solicita http://localhost:5000. Docker redirige el puerto del equipo al puerto 5000 del servicio web. Cuando el usuario crea o elimina una tarea, Flask utiliza la biblioteca redis para enviar la operación a datos:6379. Al consultar la página, lee los registros y genera la respuesta HTML.

Comunicación entre servicios

La variable REDIS_HOST tiene el valor datos. Compose permite resolver ese nombre en la red compartida. No se utiliza localhost para acceder al almacenamiento desde web, porque localhost dentro de un contenedor se refiere a ese mismo contenedor. Redis no publica su puerto hacia el equipo anfitrión (Docker, s. f.-a).

Inicio y persistencia

El servicio web depende de que la comprobación redis-cli ping del servicio datos sea satisfactoria. Esta condición ordena el inicio de la composición (Docker, s. f.-b). Redis utiliza AOF y guarda sus archivos en /data, directorio asociado al volumen tareas. Detener y recrear los contenedores debe conservar la información mientras no se elimine el volumen.

Implementación y ejecución

Archivo	Contenido
app.py	Rutas HTTP, validación y operaciones con Redis.
templates/index.html	Formulario y lista de tareas.
Dockerfile	Construcción de la imagen y arranque con Gunicorn.
compose.yaml	Dos servicios, red, volumen y comprobaciones de salud.
requirements.txt	Versiones de Flask, redis y Gunicorn.
comprobar.py	Nueve comprobaciones automáticas sobre HTTP y Redis.
INICIAR.bat y VERIFICAR.bat	Inicio y generación de registros en Windows.

Construcción

Con Docker Desktop iniciado, se abre PowerShell en la carpeta que contiene compose.yaml y se ejecuta:

```
docker compose up --build -d --wait
docker compose ps
```

La opción --build construye la imagen web. El servicio datos usa la imagen oficial de Redis. Aunque se construye una imagen propia, la composición inicia dos contenedores, que es el requisito de la actividad.

Uso y comprobación

Se accede a http://localhost:5000, se registran dos tareas y se consulta http://localhost:5000/salud. Para comprobar la lectura de los mismos registros desde ambos servicios:

```
docker compose exec -T web python comprobar.py
docker compose exec -T datos redis-cli HGETALL tareas
```

Comprobación del volumen

```
docker compose down
docker compose up -d --wait
```

Después se vuelve a consultar la aplicación y se verifica que las tareas permanezcan. No se usa down -v, porque elimina el volumen. Los archivos BAT incluidos permiten iniciar, comprobar y detener el proyecto con doble clic.

Resultados de las pruebas realizadas

Se ejecutó la aplicación con Gunicorn y un servidor Redis real como procesos locales. Las solicitudes HTTP permitieron verificar las operaciones y contrastarlas con una lectura directa del almacenamiento. Estas pruebas se realizaron fuera de Docker.

Prueba	Resultado
Interfaz HTTP disponible	Aprobada
Conexion real con Redis	Aprobada
Rechazo de tarea vacia	Aprobada
Rechazo de mas de 120 caracteres	Aprobada
Creacion por HTTP	Aprobada
Lectura directa en Redis del registro creado	Aprobada
Consulta visible en interfaz	Aprobada
Escape de HTML en interfaz	Aprobada
Eliminacion por HTTP	Aprobada
Conservación de datos tras reiniciar Redis local	Aprobada

El ensayo de persistencia conservó los dos registros después de detener y volver a iniciar Redis con AOF. El archivo persistencia_redis_real.json registra dos elementos antes y después y la igualdad de los datos.

Entorno y límites de la evidencia

La prueba local utilizó Python 3.12, Gunicorn 23.0.0 y Redis 6.2.14. La composición solicita Redis 7.4 Alpine; por ello, el resultado local no valida esa imagen específica. Tampoco comprueba la red ni el volumen de Docker.

En el entorno de preparación no fue posible iniciar el motor Docker. La construcción de la imagen y la ejecución real de los dos contenedores deben respaldarse con los registros de Docker Desktop o con una ejecución aprobada del flujo de GitHub Actions. No se presentan estos resultados como si la composición ya se hubiera ejecutado.

Archivos de respaldo

La carpeta evidencias contiene las nueve comprobaciones en JSON, los registros de tareas, la comparación de persistencia y dos capturas de la aplicación local. LEEME.md explica la procedencia de cada archivo.

Evidencia de la aplicación local

La siguiente captura corresponde a una ejecución real de la aplicación con dos tareas guardadas en Redis. La interfaz es la misma que se sirve con la configuración de Docker; esta captura, por sí sola, no demuestra el uso de contenedores.

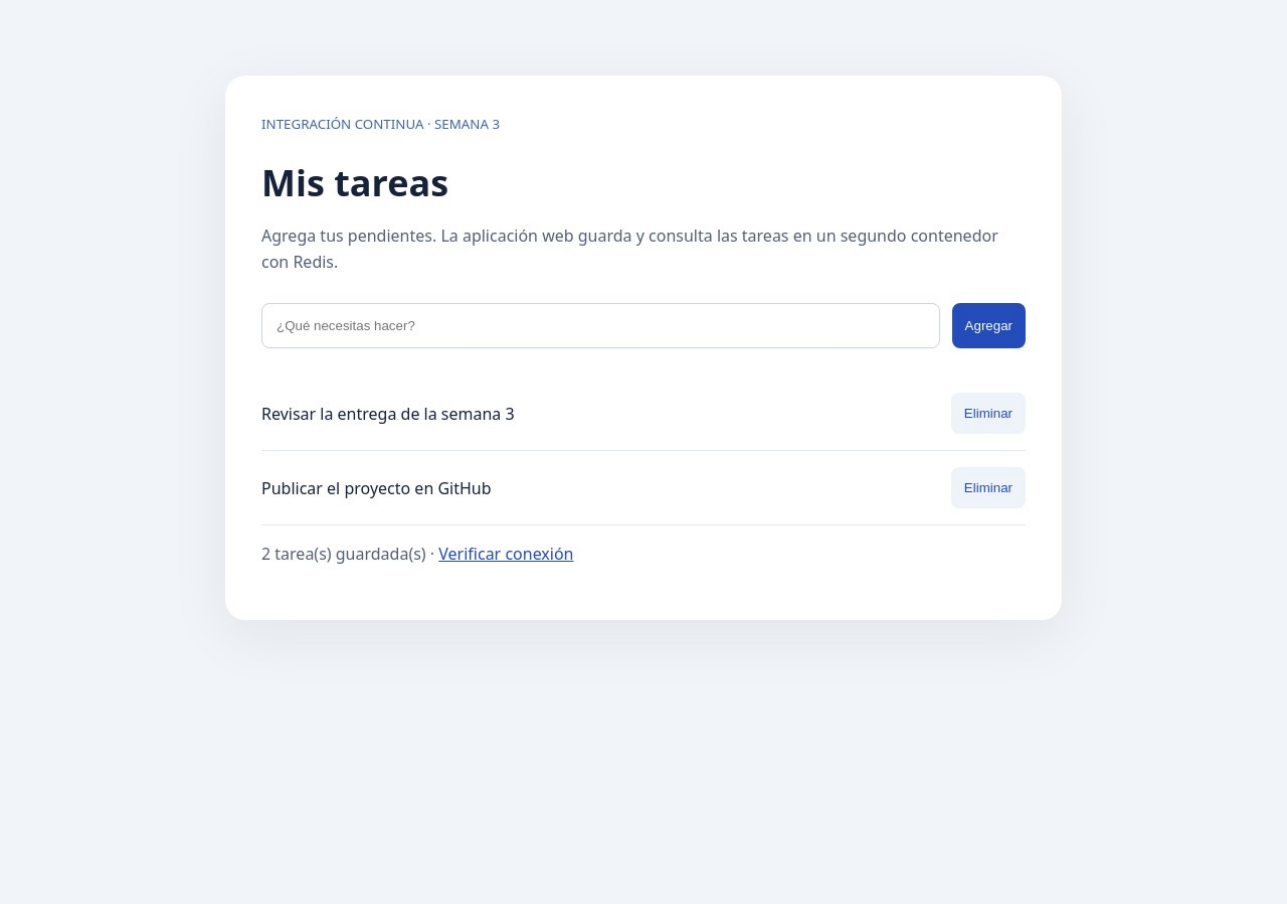


Figura 1. Aplicación ejecutada localmente con Redis real. Fuente: ejecución del proyecto.

Consulta del estado

La ruta /salud realiza una comprobación de Redis y devuelve el número de tareas. Durante la ejecución respondió con estado ok, conexion_datos ok y dos tareas guardadas. La captura completa está disponible en salud_ejecucion_local.png.

Lectura directa del almacenamiento

Los títulos guardados fueron “Revisar la entrega de la semana 3” y “Publicar el proyecto en GitHub”. Se verificaron directamente en el hash tareas de Redis, además de observarlos en la respuesta de la aplicación.

Publicación y comprobación en GitHub

El código se publicó en una rama independiente del repositorio DevelopingGeek de la cuenta xChichoTop. La carpeta entrega-1-semana-3 contiene esta actividad; la rama master conserva el proyecto anterior.

Repositorio y carpeta de la entrega:
https://github.com/xChichoTop/DevelopingGeek/tree/entrega-1-semana-3/entrega-1-semana-3

Rama: entrega-1-semana-3
Commit inicial de la entrega: 5fde33fc69bc4e772845984ddfd612546822ee7f

Comprobación automática preparada

El flujo Comprobar dos contenedores Docker define una ejecución en Linux que valida Compose, construye e inicia los servicios, ejecuta comprobar.py desde web y lee los datos desde Redis. Después recrea los contenedores y comprueba que el registro de prueba continúe en el volumen.

El flujo también guarda construcción, estado, pruebas y persistencia en un artefacto llamado evidencias-dos-contenedores. El flujo quedó registrado en GitHub, pero la primera ejecución no llegó a iniciar los pasos de construcción. No existe todavía una ejecución aprobada.

Evidencias que completan la validación Docker

1. Registro de construcción sin errores y estado de los servicios web y datos.
2. Resultado de comprobar.py ejecutado dentro de web, con conexión al servicio datos.
3. Lectura de Redis y conservación de la tarea después de recrear los contenedores.
4. Enlace de una ejecución aprobada en Actions o capturas equivalentes obtenidas en Docker Desktop.

Se generó la ejecución de GitHub Actions 42946871111, pero el servicio no inició el runner y no ejecutó ningún paso. Por tanto, Docker continúa pendiente de validación. Enlace de diagnóstico: https://github.com/xChichoTop/DevelopingGeek/actions/runs/42946871111. Se debe completar la comprobación en Docker Desktop o resolver la restricción de la cuenta y repetir el flujo.

Conclusiones

La aplicación implementa las operaciones previstas y cuenta con una configuración para separar web y almacenamiento en dos contenedores. La consulta de datos usa el nombre del servicio Redis, de modo que la comunicación prevista depende de la red compartida declarada en Compose.

Las pruebas HTTP con un servidor Redis real confirmaron la creación, consulta y eliminación de tareas, así como la validación de entradas y el escape de HTML. El ensayo local de reinicio confirmó la conservación de dos registros con AOF.

El código está publicado en GitHub. Para cerrar el requisito práctico de la entrega hace falta confirmar una ejecución de Docker que demuestre ambos contenedores y su comunicación. El proyecto incluye instrucciones y una comprobación automática para obtener esa evidencia sin sustituirla por las pruebas locales.

Referencias

Politécnico Grancolombiano. (s. f.). Proyecto de software basado en herramientas de integración continua [Guía de proyecto]. Entrega 1, semana 3, p. 3.

Oliveros Acosta, E. E. (s. f.). Máquinas virtuales y Dockers para construcción de ambientes [Lectura fundamental, escenario 3]. Politécnico Grancolombiano.

Docker. (s. f.-a). Networking in Compose. https://docs.docker.com/compose/how-tos/networking/

Docker. (s. f.-b). Control startup and shutdown order in Compose. https://docs.docker.com/compose/how-tos/startup-order/

Datos administrativos por completar

Antes de presentar el informe, completar únicamente los nombres de integrantes y docente en la primera página. El enlace de GitHub corresponde al proyecto publicado. El estado de las pruebas Docker debe actualizarse cuando exista evidencia de ejecución.